



Unit-02

Object Oriented Programming



Difference between Procedure Oriented and Object Oriented Programming

- ❖ Procedural programming creates a step by step program that guides the application through a sequence of instructions. Each instruction is executed in order.
- ❖ Procedural programming also focuses on the idea that all algorithms are executed with functions and data that the programmer has access to and is able to change.
- ❖ Object-Oriented programming is much more similar to the way the real world works; it is analogous to the human brain. Each program is made up of many entities called objects.
- ❖ Instead, a message must be sent requesting the data, just like people must ask one another for information; we cannot see inside each other's heads.

Summarized the differences

Procedure Oriented Programming	Object Oriented Programming
In POP, program is divided into small parts called functions	In OOP, program is divided into parts called objects .
POP follows Top Down approach	OOP follows Bottom Up approach
POP does not have any proper way for hiding data so it is less secure .	OOP provides Data Hiding so provides more security
In POP, Overloading is not possible	In OOP, overloading is possible in the form of Function Overloading and Operator Overloading
In POP, Most function uses Global data for sharing that can be accessed freely from function to function in the system.	In OOP, data can not move easily from function to function, it can be kept public or private so we can control the access of data.
POP does not have any access specifier	OOP has access specifiers named Public, Private, Protected, etc.

Featuers of OOP

- ❖ Ability to simulate real-world event much more effectively
- ❖ Code is reusable thus less code may have to be written
- ❖ Data becomes active
- ❖ Better able to create GUI (graphical user interface) applications
- ❖ Programmers are able to produce faster, more accurate and better-written applications

Fundamental concepts of OOP in Python

The four major principles of object orientation are:

- ❖ Encapsulation
- ❖ Data Abstraction
- ❖ Inheritance
- ❖ Polymorphism

What is an Object..?

- ❖ Objects are the basic run-time entities in an object-oriented system.
- ❖ They may represent a person, a place, a bank account, a table of data or any item that the program must handle.
- ❖ When a program is executed the objects interact by sending messages to one another.
- ❖ Objects have two components:
 - Data (i.e., attributes)
 - Behaviors (i.e., methods)

Object Attributes and Methods Example

Object Attributes

- Store the data for that object
- Example (taxi):
 - Driver
 - OnDuty
 - NumPassengers
 - Location

Object Methods

- Define the behaviors for the object
- Example (taxi):
 - PickUp
 - DropOff
 - GoOnDuty
 - GoOffDuty
 - GetDriver
 - SetDriver
 - GetNumPassengers

What is a Class..?

- ❖ A class is a special data type which defines how to build a certain kind of object.
- ❖ The class also stores some data items that are shared by all the instances of this class
- ❖ Instances are objects that are created which follow the definition given inside of the class
- ❖ Python doesn't use separate class interface definitions as in some languages
- ❖ You just define the class and then use it

Methods in Classes

- ❖ Define a method in a class by including function definitions within the scope of the class block
- ❖ There must be a special first argument **self** in all of method definitions which gets bound to the calling instance
- ❖ There is usually a special method called **__init__** in most classes

A Simple Class def: Student

```
class student:
    """A class representing a student """
    def __init__(self, n, a):
        self.full_name = n
        self.age = a
    def get_age(self):      #Method
        return self.age
```

❖ Define class:

- Class name, begin with capital letter, by convention
- object: class based on (Python built-in type)

❖ Define a method

- Like defining a function
- Must **have a special first parameter**, self, which provides way for a method to refer to object itself

Instantiating Objects with ‘__init__’

- ❖ __init__ is the default constructor
- ❖ __init__ serves as a constructor for the class. Usually does some initialization work
- ❖ An __init__ method can take any number of arguments
- ❖ However, the first argument self in the definition of __init__ is special

Self

- ❖ The first argument of every method is a reference to the current instance of the class
- ❖ By convention, we name this argument **self**
- ❖ In `__init__`, `self` refers to the object currently being created; so, in other class methods, it refers to the instance whose method was called
- ❖ Similar to the keyword `this` in Java or C++
- ❖ But Python uses `self` more often than Java uses `this`
- ❖ You **do not** give a value for this parameter(`self`) when you call the method, Python will provide it.

Continue...

...Continue

- ❖ Although you must specify `self` explicitly when defining the method, you don't include it when calling the method.
- ❖ Python passes it for you automatically

Defining a method:

(this code inside a class definition.)

```
def get_age(self, num):  
    self.age = num
```

Calling a method:

```
>>> x.get_age(23)
```

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("ABC", 36)

print(p1.name)
print(p1.age)
```

Output:

ABC

36


```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def displayEmployee(self):
        print("Name : ", self.name, ", Salary: ", self.salary)

"This would create first object of Employee class"
emp1 = Employee("ABC", 2000)
"This would create second object of Employee class"
emp2 = Employee("DEF", 5000)
emp1.displayEmployee()
emp2.displayEmployee()
```

Output:

Name : ABC , Salary: 2000

Name : DEF , Salary: 5000

```
1 class add:
2     def addition(self, a, b):
3         return a+b
4
5 print("Enter Two Numbers: ", end="")
6 nOne = int(input())
7 nTwo = int(input())
8
9 cobj = add()
10 res = cobj.addition(nOne, nTwo)
11
12 print("\n" +str(nOne)+ " + " +str(nTwo)+ " = " +str(res))
```

Output:

Enter Two Numbers: 12

34

12 + 34 = 46

Deleting instances: No Need to “free”

- ❖ When you are done with an object, you don't have to delete or free it explicitly.
- ❖ Python has automatic garbage collection.
- ❖ Python will automatically detect when all of the references to a piece of memory have gone out of scope. Automatically frees that memory.
- ❖ Generally works well, few memory leaks
- ❖ There's also no “destructor” method for classes

Syntax for accessing attributes and methods

```
>>> f = student("Python", 14)
```

```
>>> f.full_name # Access attribute  
"Python"
```

```
>>> f.get_age() # Access a method  
14
```


Simple Program for Class and Object

```
class Student:
```

```
# Initializing the variables
```

```
    def __init__(self,name,age):
```

```
        self.fullname=name
```

```
        self.sage=age
```

```
# Display the entered data such as Students name and age
```

```
    def display(self):
```

```
        print 'Student FullName: %s' %(self.fullname)
```

```
        print 'Stuent Age: %d'%(self.sage)
```

```
# S1,S2 and S3 objects
```

```
# S1,S2 and S3 are three different student details and age
```

```
s1=Student('Python',14)
```

```
s1.display()
```

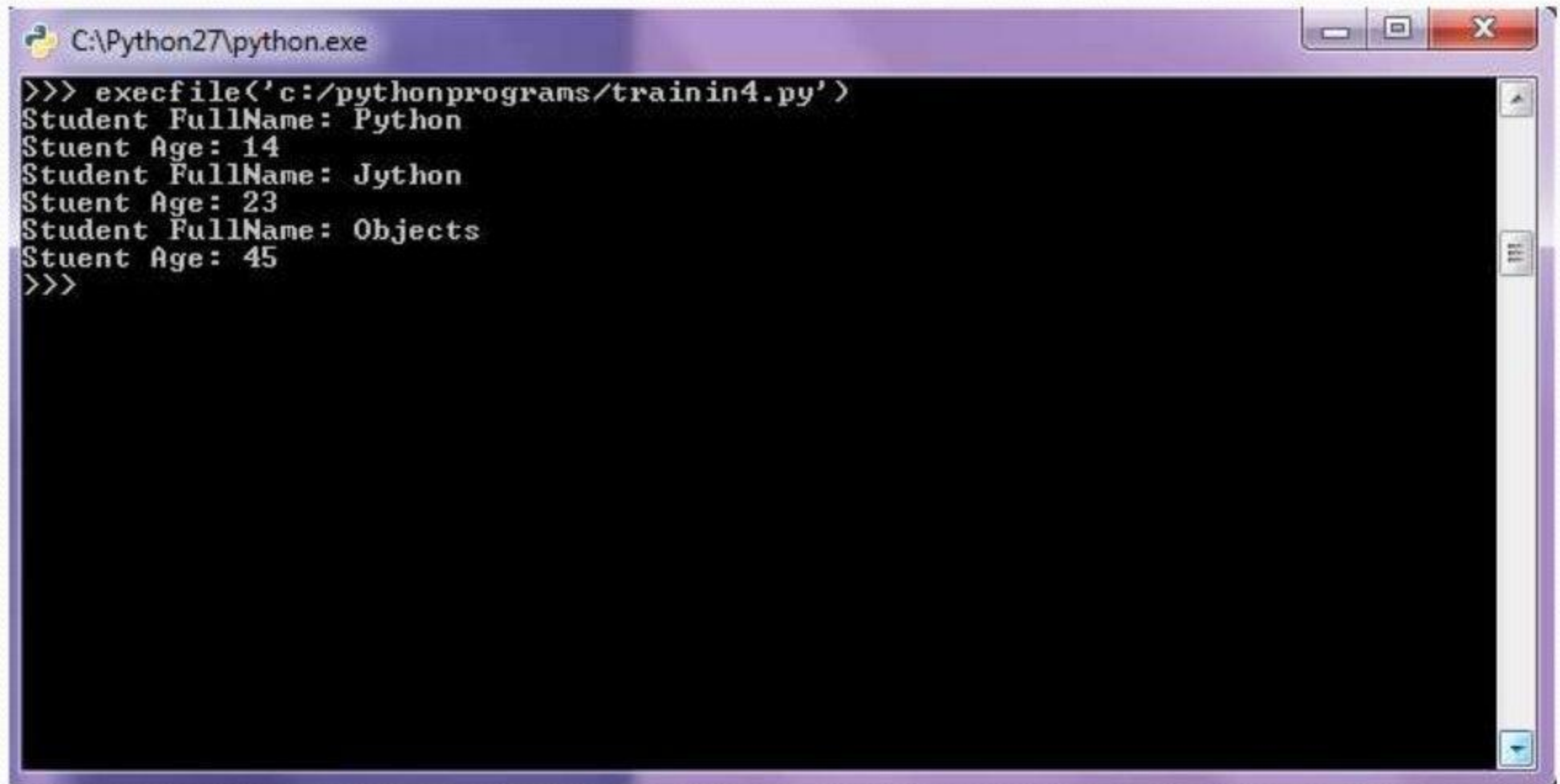
```
s2=Student('Jython',23)
```

```
s2.display()
```

```
s3=Student('Objects',45)
```

```
s3.display()
```

Output for the Sample Code



A screenshot of a Windows command prompt window titled "C:\Python27\python.exe". The window has a standard Windows interface with minimize, maximize, and close buttons in the top right corner. The command prompt shows the execution of a Python script using the `execfile` function. The output of the script is displayed in a monospaced font, showing three lines of student information: "Student FullName: Python", "Stuent Age: 14", "Student FullName: Jython", "Stuent Age: 23", "Student FullName: Objects", and "Stuent Age: 45". The prompt ends with three greater-than signs (`>>>`).

```
>>> execfile('c:/pythonprograms/trainin4.py')
Student FullName: Python
Stuent Age: 14
Student FullName: Jython
Stuent Age: 23
Student FullName: Objects
Stuent Age: 45
>>>
```


Constructors in Python

- The task of constructors is to initialize(assign values) to the data members of the class when an object of the class is created.
- In Python the `__init__()` method is called the constructor and is always called when an object is created.
- Syntax of constructor declaration :

```
def __init__(self):  
    # body of the constructor
```

- Types of constructors :
- **default constructor:** The default constructor is a simple constructor which doesn't accept any arguments. Its definition has only one argument which is a reference to the instance being constructed.
- **parameterized constructor:** constructor with parameters is known as parameterized constructor.

Default Constructors in Python

```
class constru:
    # Constructor - non parameterized
    def __init__(self):
        print("This is non parametrized constructor")
    def show(self,name):
        print("Hello",name)
Con=constru()
Con.show("John")
```

Output:-

This is non parametrized constructor
Hello John

Parameterised Constructors in Python

```
1 class Param:
2     # Constructor - parameterized
3     def __init__(self, name):
4         print("This is parametrized constructor")
5         self.name = name
6     def show(self):
7         print("Hello", self.name)
8 p1=Param("John")
9 p1.show()
```

Output:-

This is parametrized constructor
Hello John

Destructors in Python

Destructors are called when an object gets destroyed. In Python, destructors are not needed as much as in C++ because Python has a garbage collector that handles memory management automatically.

The `__del__()` method is known as a destructor method in Python. It is called when all references to the object have been deleted i.e when an object is garbage collected.

Syntax of destructor declaration :

```
def __del__(self):  
    # body of destructor
```

Note : A reference to objects is also deleted when the object goes out of reference or when the program ends.

Destructors in Python

```
# Python program to illustrate destructor
class Employee:

    # Initializing
    def __init__(self):
        print('Employee created.')

    # Deleting (Calling destructor)
    def __del__(self):
        print('Destructor called, Employee deleted.')

obj = Employee()
del obj
```

Output:

```
Employee created.
Destructor called, Employee deleted.
```


Destructors in Python



```
# Python program to illustrate destructor

class Employee:

    # Initializing
    def __init__(self):
        print('Employee created')

    # Calling destructor
    def __del__(self):
        print("Destructor called")

def Create_obj():
    print('Making Object...')
    obj = Employee()
    print('function end...')
    return obj

print('Calling Create_obj() function...')
obj = Create_obj()
print('Program End...')
```

```
Calling Create_obj() function...
Making Object...
Employee created
function end...
Program End...
Destructor called
```

Encapsulation

- ❖ Important advantage of OOP consists in the encapsulation of data. We can say that object-oriented programming relies heavily on encapsulation.
- ❖ The terms encapsulation and abstraction (also data hiding) are often used as synonyms. They are nearly synonymous, i.e. abstraction is achieved through encapsulation.
- ❖ Data hiding and encapsulation are the same concept, so it's correct to use them as synonyms
- ❖ Generally speaking encapsulation is the mechanism for restricting the access to some of an object's components, this means, that the internal representation of an object can't be seen from outside of the object's definition.

- ❖ Access to this data is typically only achieved through special methods: *Getters* and *Setters*
- ❖ By using solely get() and set() methods, we can make sure that the internal data cannot be accidentally set into an inconsistent or invalid state.
- ❖ C++, Java, and C# rely on the public, private, and protected keywords in order to implement variable scoping and encapsulation
- ❖ It's nearly always possible to circumvent this protection mechanism

Public, Protected and Private Data

- ❖ If an identifier doesn't start with an underscore character "_" it can be accessed from outside, i.e. the value can be read and changed
- ❖ Data can be protected by making members private or protected. Instance variable names starting with two underscore characters cannot be accessed from outside of the class.
- ❖ At least not directly, but they can be accessed through private name mangling.
- ❖ That means, private data `__A` can be accessed by the following name construct: *`instance_name._classname__A`*

- ❖ If an identifier is only preceded by one underscore character, it is a protected member.
- ❖ Protected members can be accessed like public members from outside of class

Example:

```
class Encapsulation(object):  
    def __init__(self, a, b, c):  
        self.public = a  
        self._protected = b  
        self.__private = c
```


The following interactive sessions shows the behavior of public, protected and private members:

```
>>> x = Encapsulation(11,13,17)
```

```
>>> x.public
```

```
11
```

```
>>> x._protected
```

```
13
```

```
>>> x._protected = 23
```

```
>>> x._protected
```

```
23
```

```
>>> x.__private
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
AttributeError: 'Encapsulation' object has no attribute '__private'
```

```
>>>
```


The following table shows the different behavior Public, Protected and Private Data

Name	Notation	Behavior
name	Public	Can be accessed from inside and outside
_name	Protected	Like a public member, but they shouldn't be directly accessed from outside
__name	Private	Can't be seen and accessed from outside

Encapsulation Example

Python provides access to all the variables and methods globally.

```
class Person:
    def __init__(self, name, age=0):
        self.name = name
        self.age = age

    def display(self):
        print(self.name)
        print(self.age)

person = Person('Dev', 30)
#accessing using class method
person.display()
#accessing directly from outside
print(person.name)
print(person.age)
```

Output

```
Dev
30
Dev
30
```

Encapsulation Example

Python offers some sort of support to the private modifier. This mechanism is called **Name mangling**. With this, it is still possible to access the class members from outside it. In Python, any identifier with `__Var` is rewritten by a python interpreter as `_Classname__Var`, and the class name remains as the present class name. This mechanism of changing names is called **Name Mangling** in Python.

```
class Person:
    def __init__(self, name, age=0):
        self.name = name
        self.__age = age

    def display(self):
        print(self.name)
        print(self.__age)

person = Person('Dev', 30)
#accessing using class method
person.display()
#accessing directly from outside
print('Trying to access variables from outside the class ')
print(person.name)
print(person.__age)
```

Output

```
Dev
30
Trying to access variables from outside the class
Dev
Traceback (most recent call last):
  File "Person.py", line 16, in <module>
    print(person.__age)
AttributeError: 'Person' object has no attribute '__age'
```


Using Getter and Setter methods to access private variables

```
class Person:
    def __init__(self, name, age=0):
        self.name = name
        self.__age = age

    def display(self):
        print(self.name)
        print(self.__age)

    def getAge(self):
        print(self.__age)

    def setAge(self, age):
        self.__age = age

person = Person('Dev', 30)
#accessing using class method
person.display()
#changing age using setter
person.setAge(35)
person.getAge()
```

Output

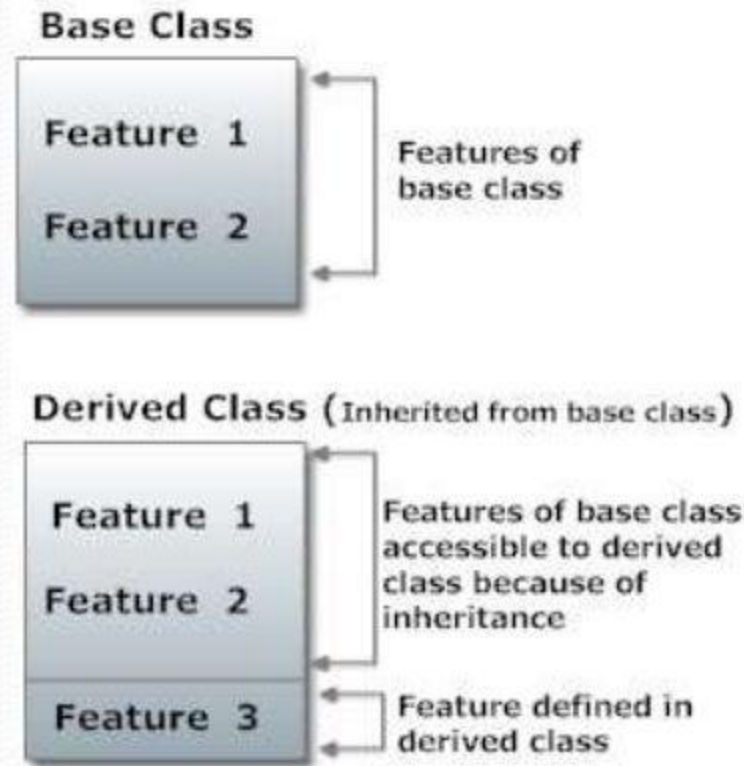
```
Dev
30
35
```

Inheritance

- ❖ Inheritance is a powerful feature in object oriented programming
- ❖ It refers to defining a new class with little or no modification to an existing class.
- ❖ The new class is called derived (or child) class and the one from which it inherits is called the base (or parent) class.
- ❖ Derived class inherits features from the base class, adding new features to it.
- ❖ This results into re-usability of code.

Syntax:

```
class Baseclass(Object):  
    body_of_base_class  
class DerivedClass(BaseClass):  
    body_of_derived_class
```



While designing a inheritance concept, following key pointes keep it in mind

- ❖ A sub type never implements less functionality than the super type
- ❖ Inheritance should never be more than two levels deep
- ❖ We use inheritance when we want to avoid redundant code.



Two built-in functions *isinstance()* and *issubclass()* are used to check inheritances.

- ❖ Function *isinstance()* returns True if the object is an instance of the class or other classes derived from it.
- ❖ Each and every class in Python inherits from the base class object.

Single Inheritance Example

```
class Animal:
    def speak(self):
        print("Animal Speaking")
#child class Dog inherits the base class Animal
class Dog(Animal):
    def bark(self):
        print("dog barking")
d = Dog()
d.bark()
d.speak()
```

Output:

```
dog barking
Animal Speaking
```

Multilevel Inheritance Example

Syntax

```
class class1:
    <class-suite>
class class2(class1):
    <class suite>
class class3(class2):
    <class suite>
```

```
class Animal:
    def speak(self):
        print("Animal Speaking")
#The child class Dog inherits the base class Animal
class Dog(Animal):
    def bark(self):
        print("dog barking")
#The child class Dogchild inherits another child class Dog
class DogChild(Dog):
    def eat(self):
        print("Eating bread...")
d = DogChild()
d.bark()
d.speak()
d.eat()
```

Output:

```
dog barking
Animal Speaking
Eating bread...
```


Multiple Inheritance Example

Syntax

```
class Base1:
```

```
    <class-suite>
```

```
class Base2:
```

```
    <class-suite>
```

```
.
```

```
.
```

```
.
```

```
class BaseN:
```

```
    <class-suite>
```

```
class Derived(Base1, Base2, ..... BaseN):
```

```
    <class-suite>
```

```
class Calculation1:
```

```
    def Summation(self,a,b):
```

```
        return a+b;
```

```
class Calculation2:
```

```
    def Multiplication(self,a,b):
```

```
        return a*b;
```

```
class Derived(Calculation1,Calculation2):
```

```
    def Divide(self,a,b):
```

```
        return a/b;
```

```
d = Derived()
```

```
print(d.Summation(10,20))
```

```
print(d.Multiplication(10,20))
```

```
print(d.Divide(10,20))
```

Output:

```
30
```

```
200
```

```
0.5
```

The `issubclass(sub,sup)` method

```
class Calculation1:
    def Summation(self,a,b):
        return a+b;
class Calculation2:
    def Multiplication(self,a,b):
        return a*b;
class Derived(Calculation1,Calculation2):
    def Divide(self,a,b):
        return a/b;
d = Derived()
print(issubclass(Derived,Calculation2))
print(issubclass(Calculation1,Calculation2))
```

Output:

```
True
False
```

Method Overriding

When the parent class method is defined in the child class with some specific implementation, then the concept is called method overriding.

```
class Bank:
    def getroi(self):
        return 10;
class SBI(Bank):
    def getroi(self):
        return 7;

class ICICI(Bank):
    def getroi(self):
        return 8;
b1 = Bank()
b2 = SBI()
b3 = ICICI()
print("Bank Rate of interest:",b1.getroi());
print("SBI Rate of interest:",b2.getroi());
print("ICICI Rate of interest:",b3.getroi());
```

Output:

```
Bank Rate of interest: 10
SBI Rate of interest: 7
ICICI Rate of interest: 8
```


Sample code for Inheritance

```
class Person(object):
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def show(self):
        print 'Person Name: %s' %(self.name)
        print 'Person Age: %s' %(self.age)
class Employee(Person):
    def __init__(self,name,age,company,sal):
        self.company=company
        self.sal=sal
        super(Employee,self).__init__(name,age)
    def __repr__(self):
        return str
        (self.name+self.age+self.company+self.sal)
```

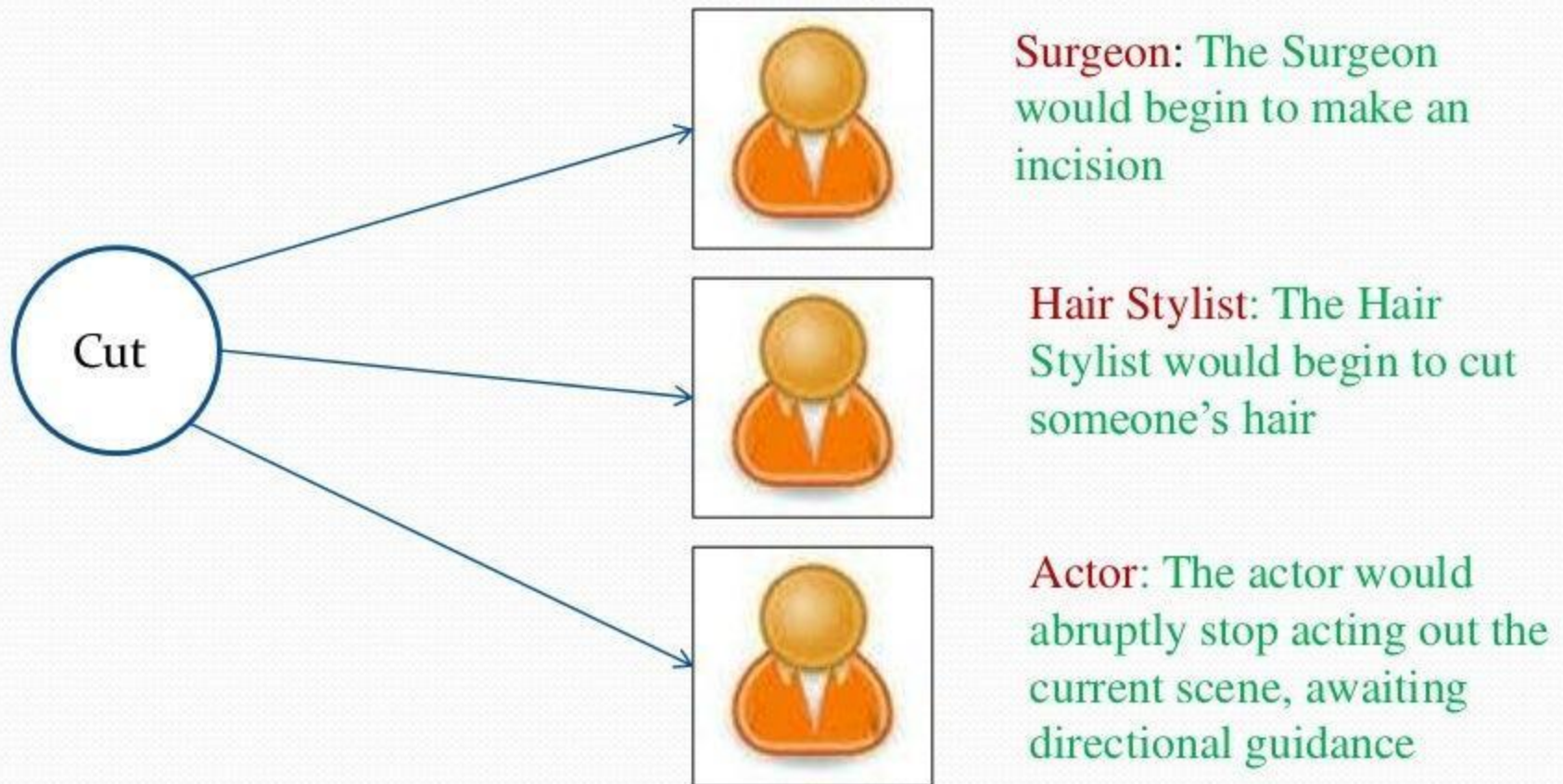
```
def showme(self):
    super(Employee,self).show()
    print 'Company Name: %s'%(self.company)
    print 'Employee Salary per Annum: %s' %(self.sal)
    print '\n'
empdict={'guido':['45','PYTHON','500000'],
        'van':['25','JYTHON','200000'],
        'rosum':['35','ABC','400000']}
for key,value in empdict.iteritems():
    print key,value
    emp=Employee(key,value[0],value[1],value[2])
    emp.showme()
```

Polymorphism

- ❖ Polymorphism in Latin word which made up of '*ploy*' means many and '*morphs*' means forms
- ❖ From the Greek , Polymorphism means *many(poly)* *shapes (morph)*
- ❖ This is something similar to a word having several different meanings depending on the context
- ❖ Generally speaking, polymorphism means that a method or function is able to cope with different types of input.

A simple word 'Cut' can have different meaning depending where it is used

If any body says "Cut" to these people



In OOP , Polymorphism is the characteristic of being able to assign a different meaning to a particular symbol or operator in different contexts specifically to allow an entity such as a variable, a function or an object to have more than one form.

There are two kinds of Polymorphism

Overloading :

Two or more methods with different signatures

Overriding:

Replacing an inherited method with another having the same signature

A Sample Program for Polymorphism

```
class Person:
    def __init__(self, name): # Constructor of the class
        self.name = name
    def designation(self):     # Abstract method, defined by convention only
        raise NotImplementedError("Subclass must implement abstract method")

class Employee(Person):
    def designation(self):
        return 'Software Engineer'

class Doctor(Person):
    def designation(self):
        return 'Cardiologist'

class Student(Person):
    def designation(self):
        return 'Graduate Engineer'

persons = [Employee('Guido Van Rossum'), Doctor('Chalapathi'), Student('Robert')]

for person in persons:
    print person.name + ': ' + person.designation()
```

Abstraction in Python

- Abstraction is used to hide the internal functionality of the function from the users.
- In Python, abstraction can be achieved by using abstract classes and interfaces.
- A class that consists of one or more abstract method is called the abstract class. Abstract methods do not contain their implementation. Abstract class can be inherited by the subclass and abstract method gets its definition in the subclass.
- By default, Python does not provide abstract classes. Python comes with a module that provides the base for defining Abstract Base classes(ABC) and that module name is ABC.
- **ABC** works by decorating methods of the base class as abstract and then registering concrete classes as implementations of the abstract base.
- A method becomes abstract when decorated with the keyword `@abstractmethod`.

Abstraction in Python

```
from abc import ABC, abstractmethod
```

```
class Polygon(ABC):
```

```
    @abstractmethod
```

```
    def noofsides(self):
```

```
        pass
```

```
class Triangle(Polygon):
```

```
    # overriding abstract method
```

```
    def noofsides(self):
```

```
        print("I have 3 sides")
```

```
class Pentagon(Polygon):
```

```
    # overriding abstract method
```

```
    def noofsides(self):
```

```
        print("I have 5 sides")
```

```
class Hexagon(Polygon):
```

```
    # overriding abstract method
```

```
    def noofsides(self):
```

```
        print("I have 6 sides")
```

```
class Quadrilateral(Polygon):
```

```
    # overriding abstract method
```

```
    def noofsides(self):
```

```
        print("I have 4 sides")
```

```
# Driver code
```

```
R = Triangle()
```

```
R.noofsides()
```

```
K = Quadrilateral()
```

```
K.noofsides()
```

```
R = Pentagon()
```

```
R.noofsides()
```

```
K = Hexagon()
```

```
K.noofsides()
```

Output:

```
I have 3 sides
```

```
I have 4 sides
```

```
I have 5 sides
```

```
I have 6 sides
```

Abstract Data Type using Object--Oriented Programming in Python

ADT

Classes are the Python representation for “Abstract Data Types,” (ADT) a very useful notion in any programming language.

An ADT involves both data and operations on that data.

Example: Stack

A stack is a LIFO (last in, first out) list with the following operations: Push, Pop, Create (Init), Top, IsEmpty, Size. (Such a list is called a Signature or the Interface to the ADT.)

Stack ADT

Stack() creates a new stack that is empty. It needs no parameters and returns an empty stack.

push(item) adds a new item to the top of the stack. It needs the item and returns nothing.

pop() removes the top item from the stack. It needs no parameters and returns the item. The stack is modified.

top() returns the top item from the stack but does not remove it. It needs no parameters. The stack is not modified.

isEmpty() tests to see whether the stack is empty. It needs no parameters and returns a boolean value.

size() returns the number of items on the stack. It needs no parameters and returns an integer.

Stack ADT

Python program to demonstrate stack implementation

```
stack = []
```

append() function to push element in the stack

```
stack.append('a')
```

```
stack.append('b')
```

```
stack.append('c')
```

```
print('Initial stack')
```

```
print(stack)
```

pop() function to pop element from stack in LIFO order

```
print('\nElements popped from stack:')
```

```
print(stack.pop())
```

```
print(stack.pop())
```

```
print(stack.pop())
```

```
print('\nStack after elements are popped:')
```

```
print(stack)
```

```
Initial stack
['a', 'b', 'c']

Elements popped from stack:
c
b
a

Stack after elements are popped:
[]
```


Stack ADT

```
stack = ["Dog", "Cat", "Rat"]
stack.append("Tiger")
stack.append("Lion")
print(stack)
```

```
# Removes the last item
print(stack.pop())
```

```
print(stack)
```

```
# Removes the last item
print(stack.pop())
```

```
print(stack)
```

```
['Dog', 'Cat', 'Rat', 'Tiger', 'Lion']
Lion
['Dog', 'Cat', 'Rat', 'Tiger']
Tiger
['Dog', 'Cat', 'Rat']
```

Example:Queue

A Queue is a FIFO (first in, first out) list with the following operations: Enqueue, Dequeue, Size, Front.

Queue ADT

Queue() creates a new queue that is empty. It needs no parameters and returns an empty queue.

Enqueue(item) adds a new item to the rear of the queue. It needs the item and returns nothing.

Dequeue() removes the item from the front of the queue. It needs no parameters and returns the item. The queue is modified.

Front() returns the front item from the queue but does not remove it. It needs no parameters. The queue is not modified.

isEmpty() tests to see whether the queue is empty. It needs no parameters and returns a boolean value.

size() returns the number of items on the queue. It needs no parameters and returns an integer.

Queue ADT as a Class

Python code to demonstrate Implementing Queue using list

```
queue = ["Dog", "Cat", "Rat"]
```

```
queue.append("Tiger")
```

```
queue.append("Lion")
```

```
print(queue)
```

Removes the first item

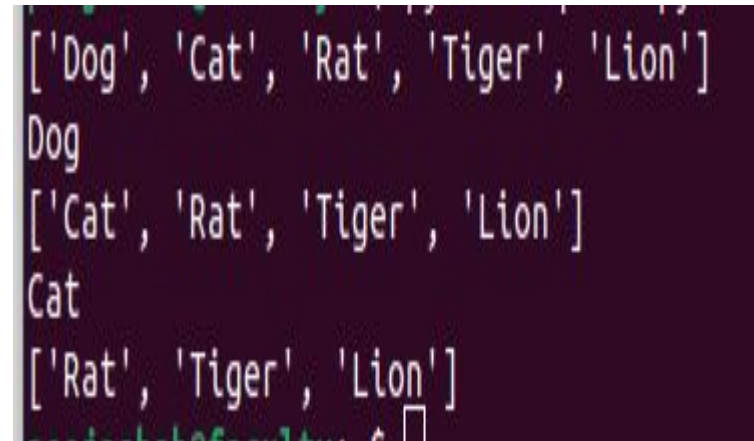
```
print(queue.pop(0))
```

```
print(queue)
```

Removes the first item

```
print(queue.pop(0))
```

```
print(queue)
```



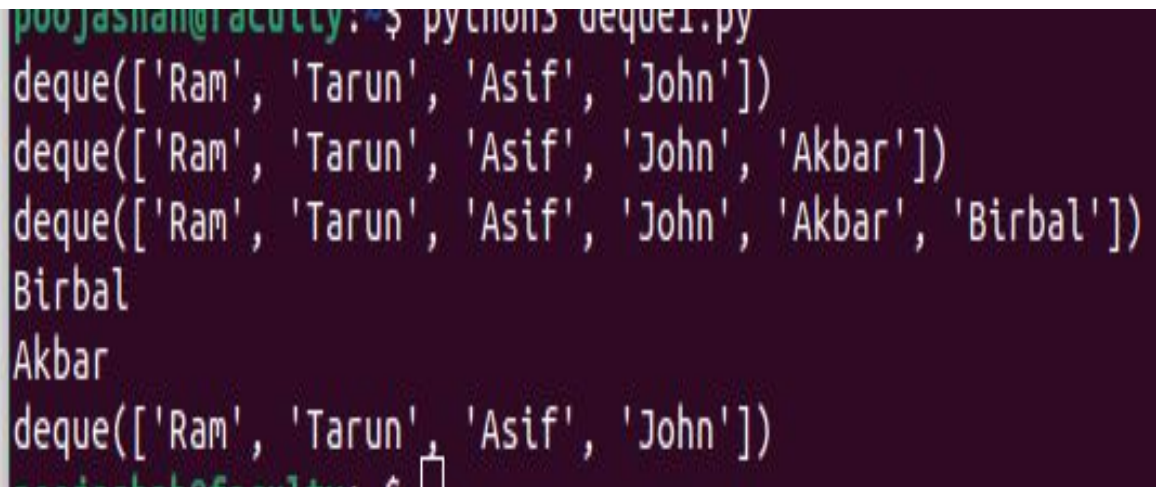
```
['Dog', 'Cat', 'Rat', 'Tiger', 'Lion']  
Dog  
['Cat', 'Rat', 'Tiger', 'Lion']  
Cat  
['Rat', 'Tiger', 'Lion']
```

deque in stack

#Python code to demonstrate Implementing Stack using deque

```
from collections import deque
queue = deque(["Ram", "Tarun", "Asif", "John"])
print(queue)
queue.append("Akbar")
print(queue)
queue.append("Birbal")
print(queue)
print(queue.pop())
print(queue.pop())
print(queue)
```

we prefer the use of deque over list, which was specially designed to have fast appends and pops from both the front and back end.



```
poo.jashan@faculty:~$ python3 deque1.py
deque(['Ram', 'Tarun', 'Asif', 'John'])
deque(['Ram', 'Tarun', 'Asif', 'John', 'Akbar'])
deque(['Ram', 'Tarun', 'Asif', 'John', 'Akbar', 'Birbal'])
Birbal
Akbar
deque(['Ram', 'Tarun', 'Asif', 'John'])
```

deque in queue

Python code to demonstrate Implementing Queue using deque

```
from collections import deque
queue = deque(["Ram", "Tarun", "Asif", "John"])
print(queue)
queue.append("Akbar")
print(queue)
queue.append("Birbal")
print(queue)
print(queue.popleft())
print(queue.popleft())
print(queue)
```

```
deque(['Ram', 'Tarun', 'Asif', 'John'])
deque(['Ram', 'Tarun', 'Asif', 'John', 'Akbar'])
deque(['Ram', 'Tarun', 'Asif', 'John', 'Akbar', 'Birbal'])
Ram
Tarun
deque(['Asif', 'John', 'Akbar', 'Birbal'])
```


Regular Expressions

What is Regular Expression

A **Regular Expression** (RegEx) is a sequence of characters that defines a search pattern. For example,

```
^a...s$
```

The above code defines a RegEx pattern. The pattern is: **any five letter string starting** with a and ending with s.

A pattern defined using RegEx can be used to match against a string.

Expression	String	Matched?
<code>^a...s\$</code>	abs	No match
	alias	Match
	abyss	Match
	Alias	No match
	An abacus	No match

Python has a module named **re** to work with regular expression. Here's an

```
import re

pattern = '^a...s$'
test_string = 'abyss'
result = re.match(pattern, test_string)

if result:
    print("Search successful.")
else:
    print("Search unsuccessful.")
```

O/p:
Search Successful

Here, we used `re.match()` function to search pattern within the `test_string`. The method returns a match object if the search is successful. If not, it returns `None`.

Specify Pattern Using RegEx:

To specify regular expressions, metacharacters are used.

Meta characters:

Metacharacters are characters that are interpreted in a special way by a RegEx engine. Here's a list of metacharacters:

`[] . ^ * + ? { } ( ) \ |`

[] – Square brackets:

Square brackets specifies a set of characters you wish to match.

Expression	String	Matched?
[abc]	a	1 match
	ac	2 matches
	Hey Jude	No match
	abc de ca	5 matches

Above, `[abc]` will match if the string you are trying to match contains any of the a, b or c.

You can also specify a range of characters using `-` inside square brackets.

- `[a-e]` is the same as `[abcde]`.
- `[1-4]` is the same as `[1234]`.
- `[0-39]` is the same as `[01239]`.

You can complement (invert) the character set by using caret `^` symbol at the start of a square-bracket.

- `[^abc]` means any character except a or b or c.
- `[^0-9]` means any non-digit character.

. - Period

period matches any single character (except newline '\n').

Expression	String	Matched?
..	a	No match
	ac	1 match
	acd	1 match
	acde	2 matches (contains 4 characters)

^ – Caret:

The caret symbol ^ is used to check if a string starts with a certain character.

Expression	String	Matched?
^a	a	1 match
	abc	1 match
	bac	No match
^ab	abc	1 match
	acb	No match (starts with a but not followed by b)

\$\$ - Dollar

The dollar symbol is used to check if a string ends with a certain character.

Expression	String	Matched?
a\$	a	1 match
	formula	1 match
	cab	No match

+ – Plus:

The plus symbol + matches one or more occurrences of the pattern left to it.

Expression	String	Matched?
ma+n	mn	No match (no a character)
	man	1 match
	maaan	1 match
	main	No match (a is not followed by n)
	woman	1 match

? - Question Mark

The question mark symbol `?` matches **zero or one occurrence** of the pattern left to it.

Expression	String	Matched?
<code>ma?n</code>	<code>mn</code>	1 match
	<code>man</code>	1 match
	<code>maaan</code>	No match (more than one <code>a</code> character)
	<code>main</code>	No match (a is not followed by n)
	<code>woman</code>	1 match

{ } - Braces

Consider this code: `{n,m}`. This means at least `n`, and at most `m` repetitions of the pattern left to it.

Expression	String	Matched?
<code>a{2,3}</code>	<code>abc dat</code>	No match
	<code>abc daat</code>	1 match (at <code>daat</code>)
	<code>aabc daaat</code>	2 matches (at <code>aabc</code> and <code>daaat</code>)
	<code>aabc daaaat</code>	2 matches (at <code>aabc</code> and <code>daaaat</code>)

| - Alternation

Vertical bar `|` is used for alternation (or operator).

Expression	String	Matched?
<code>a b</code>	<code>cde</code>	No match
	<code>ade</code>	1 match (match at <code><u>a</u>de</code>)
	<code>acdbea</code>	3 matches (at <code><u>a</u>cd<u>b</u>ea</code>)

Here, `a|b` match any string that contains either `a` or `b`

() - Group

Parentheses `()` is used to group sub-patterns. For example, `(a|b|c)xz` match any string that matches either `a` or `b` or `c` followed by `xz`

Expression	String	Matched?
<code>(a b c)xz</code>	<code>ab xz</code>	No match
	<code>abxz</code>	1 match (match at <code>ab<u>xz</u></code>)
	<code>axz cabxz</code>	2 matches (at <code><u>axz</u>bc cab<u>xz</u></code>)

`\` - Backslash

Backslash `\` is used to escape various characters including all metacharacters. For example,

`\$a` match if a string contains `$` followed by `a`. Here, `$` is not interpreted by a RegEx engine in a special way.

`\A` - Matches if the specified characters are at the start of a string.

Expression	String	Matched?
<code>\Athe</code>	the sun	Match
	In the sun	No match

`\b` - Matches if the specified characters are at the beginning or end of a word.

Expression	String	Matched?
<code>\bfoo</code>	football	Match
	a football	Match
	afootball	No match
<code>foo\b</code>	the foo	Match
	the afoo test	Match
	the afootest	No match

`\B` - Opposite of `\b`. Matches if the specified characters are **not** at the beginning or end of a word.

Expression	String	Matched?
<code>\Bfoo</code>	football	No match
	a football	No match
	afootball	Match
<code>foo\b</code>	the foo	No match
	the afoo test	No match
	the afootest	Match

`\d` - Matches any decimal digit. Equivalent to `[0-9]`

Expression	String	Matched?
<code>\d</code>	12abc3	3 matches (at <code>12abc3</code>)
	Python	No match

`\D` - Matches any non-decimal digit. Equivalent to `[^0-9]`

Expression	String	Matched?
<code>\D</code>	1ab34"50	3 matches (at <code>1ab34"50</code>)
	1345	No match

`\s` - Matches where a string contains any whitespace character. Equivalent to `[ \t\n\r\f\v]`.

Expression	String	Matched?
<code>\s</code>	Python RegEx	1 match
	PythonRegEx	No match

`\S` - Matches where a string contains any non-whitespace character. Equivalent to `[^ \t\n\r\f\v]`.

Expression	String	Matched?
<code>\S</code>	a b	2 matches (at <code>a</code> <code>b</code>)
		No match

`\S` - Matches where a string contains any non-whitespace character. Equivalent to `[^\t\n\r\f\v]`.

Expression	String	Matched?
<code>\S</code>	a b	2 matches (at <code>a</code> <code>b</code>)
		No match

`\w` - Matches any alphanumeric character (digits and alphabets). Equivalent to `[a-zA-Z0-9_]`. By the way, underscore `_` is also considered an alphanumeric character.

Expression	String	Matched?
<code>\w</code>	12&" : ; c	3 matches (at <code>1</code> <code>2</code> <code>c</code>)
	% "> !	No match

`\W` - Matches any non-alphanumeric character. Equivalent to `[^a-zA-Z0-9_]`

Expression	String	Matched?
<code>\W</code>	1a2%c	1 match (at <code>a</code> <code>%</code>)
	Python	No match

Python RegEx

Python has a module named **re** to work with regular expressions. To use it, we need to import the module.

```
import re
```

The module defines several functions and constants to work with RegEx.

re.findall()

The `re.findall()` method returns a list of strings containing all matches.

Example 1: re.findall()

```
# Program to extract numbers from a string

import re

string = 'hello 12 hi 89. Howdy 34'
pattern = '\d+'

result = re.findall(pattern, string)
print(result)

# Output: ['12', '89', '34']
```

If the pattern is not found, `re.findall()` returns an empty list.

re.split()

The `re.split()` method splits the string where there is a match and returns a list of strings where the splits have occurred.

Example 2: re.split()

```
import re

string = 'Twelve:12 Eighty nine:89.'
pattern = '\d+'

result = re.split(pattern, string)
print(result)

# Output: ['Twelve:', ' Eighty nine:', '.']
```

Run Code >>

If the pattern is not found, `re.split()` returns a list containing the original string.

re.sub()

The syntax of `re.sub()` is:

```
re.sub(pattern, replace, string)
```

The method returns a string where matched occurrences are replaced with the content of `replace` variable.

Example 3: re.sub()

```
# Program to remove all whitespaces
import re

# multiline string
string = 'abc 12\
de 23 \n f45 6'

# matches all whitespace characters
pattern = '\s+'

# empty string
replace = ''

new_string = re.sub(pattern, replace, string)
print(new_string)

# Output: abc12de23f456
```

Run Code >>

If the pattern is not found, `re.sub()` returns the original string.

re.subn()

The `re.subn()` is similar to `re.sub()` except it returns a tuple of 2 items containing the new string and the number of substitutions made.

Example 4: re.subn()

```
# Program to remove all whitespaces
import re

# multiline string
string = 'abc 12\
de 23 \n f45 6'

# matches all whitespace characters
pattern = '\s+'

# empty string
replace = ''

new_string = re.subn(pattern, replace, string)
print(new_string)

# Output: ('abc12de23f456', 4)
```

re.search()

The `re.search()` method takes two arguments: a pattern and a string. The method looks for the first location where the RegEx pattern produces a match with the string.

If the search is successful, `re.search()` returns a match object; if not, it returns `None`.

```
match = re.search(pattern, str)
```

Example 5: re.search()

```
import re

string = "Python is fun"

# check if 'Python' is at the beginning
match = re.search('\APython', string)

if match:
    print("pattern found inside the string")
else:
    print("pattern not found")

# Output: pattern found inside the string
```

match.group()

The `group()` method returns the part of the string where there is a match.

Example 6: Match object

```
import re

string = '39801 356, 2102 1111'

# Three digit number followed by space followed by two digit number
pattern = '(\d{3}) (\d{2})'

# match variable contains a Match object.
match = re.search(pattern, string)

if match:
    print(match.group())
else:
    print("pattern not found")

# Output: 801 35
```

Our pattern `(\d{3}) (\d{2})` has two subgroups `(\d{3})` and `(\d{2})`. You can get the part of the string of these parenthesized subgroups. Here's how:

```
>>> match.group(1)
'801'

>>> match.group(2)
'35'
>>> match.group(1, 2)
('801', '35')

>>> match.groups()
('801', '35')
```

`match.start()`, `match.end()` and `match.span()`

The `start()` function returns the index of the start of the matched substring. Similarly, `end()` returns the end index of the matched substring.

```
>>> match.start()
2
>>> match.end()
8
```

The `span()` function returns a tuple containing start and end index of the matched part.

```
>>> match.span()
(2, 8)
```


match.re and match.string

The `re` attribute of a matched object returns a regular expression object. Similarly, `string` attribute returns the passed string.

```
>>> match.re
re.compile('(\d{3}) (\d{2})')

>>> match.string
'39801 356, 2102 1111'
```

Python code to check validity of mobile number (Long Code)

```
import sys
number = input()
if len(number)!=10:
    print ('invalid')
    sys.exit(0)
if number[0]=='0':
    print ('invalid')
    sys.exit(0)
for chr in number:
    if chr.isalpha():
        print ('invalid')
        break
else:
    print('Valid')
```

Check validity of Mobile Number (Shorter Code)

```
import re
number = input()
if re.match('[^0][0-9]{9}',number):
    print('valid')
else:
    print('invalid')
```

Check PAN No is valid or not?

```
import re
pan=input()
if len(pan) < 10 and len(pan) > 10 :
    print ("PAN Number should be 10 characters")
    exit
elif re.search("[^a-zA-Z0-9]",pan):
    print ("No symbols allowed, only alphanumerics")
    exit
elif re.search("[0-9]",pan[0:5]):
    print ("Invalid - 1")
    exit
elif re.search("[A-Za-z]",pan[5:9]):
    print ("Invalid - 2")
    exit
elif re.search("[0-9]",pan[-1]):
    print ("Invalid - 3")
    exit
else:
    print ("Your card "+ pan + " is valid")
```


Multithreading in python

Introduction:

- Multitasking is a process of executing multiple tasks simultaneously, we use multitasking to utilize the CPU.

- Multitasking can be achieved by two ways or classified into two types

- **Process-Based Multitasking(Multiprocessing)**
- **Thread-Based Multitasking(Multithreading)**

- **Process-Based Multitasking(Multiprocessing):**

Executing multiple tasks simultaneously, where each task is separate independent process (or) program is called as process based multitasking.

Example:

- Typing a python program in notepad
- Listening audio songs
- Download a file from internet
- The above three tasks are performed simultaneously in a system, but there is no dependence between one task and another task.
- Process based multitasking is best suitable at “Operating System” level not at programming level.

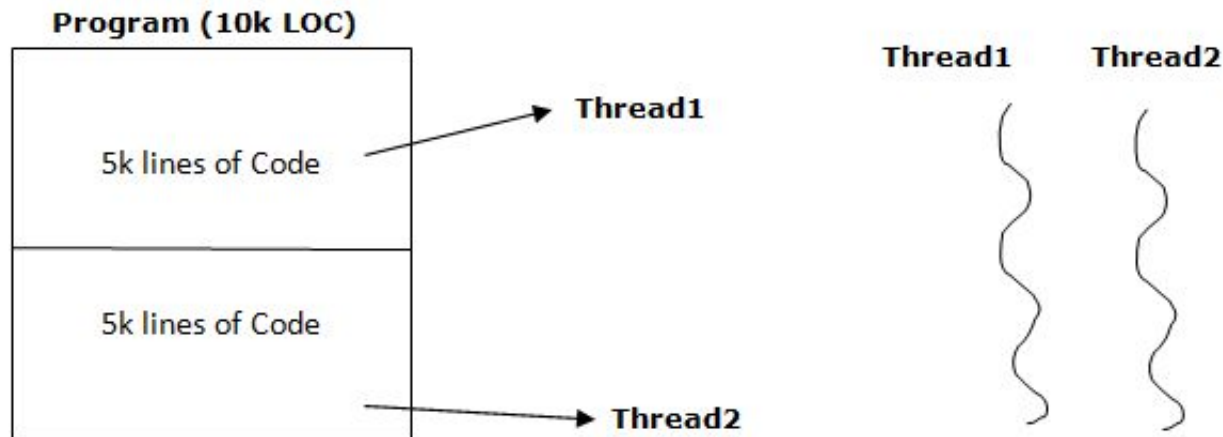
- **Thread-Based Multitasking(Multithreading):**

Executing multiple tasks simultaneously, where each task is separate independent part of process (or) program is called as thread based multitasking.

- The each independent part is called as thread. The thread based multitasking is best suitable at programming level.

Example:

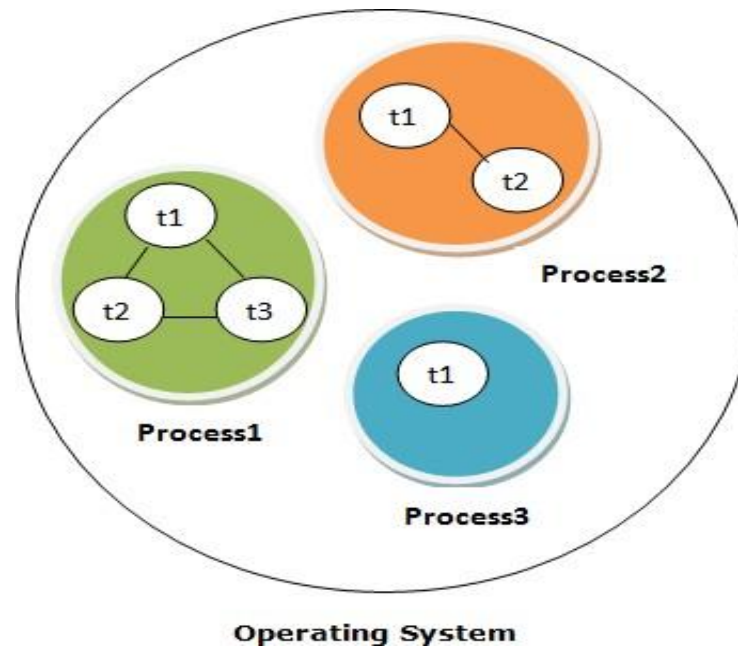
Let a program has 10k line of code, where last 5k lines of code doesn't depend on first 5k lines of code, then both are the execution simultaneously. So takes less time to complete the execution.



Note: Any type of multitasking is used to reduce response time of system and Improves performance.

Multithreading:

- A **thread** is a lightweight process; In simple words, a **thread** is a sequence of some instructions within a program that can be executed independently of other code.
- Threads are independent; if there is an exception in one thread it doesn't affect remaining threads.
- Threads shares common memory area.



- As shown in the figure, a thread is executed inside the process. There can be multiple processes inside the OS, and each process can have multiple threads.

- ❑ **Definition:** Multithreading is a process of executing multiple threads simultaneously. Multithreading allows you to break down an application into multiple sub-tasks and run these tasks simultaneously.
- In other words, the ability of a process to execute multiple threads parallelly is called multithreading. Ideally, multithreading can significantly improve the performance of any program.
 - Multiprocessing and Multithreading both are used to achieve multitasking, but we use multithreading than multiprocessing because threads share a common memory area and context-switching between threads takes less time than process.

Advantages:

- Multithreading can significantly improve the speed of computation on multiprocessor
- Multithreading allows a program to remain responsive while one thread waits for input, and another runs a GUI at the same time.

Disadvantages:

- It raises the possibility of deadlocks.
- It may cause starvation when a thread doesn't get regular access to shared resources.

❑ **Global Interpreter Lock (GIL):**

- Execution of Python code is controlled by the Python Virtual Machine.
- Python was designed in such a way that only one thread may be executing in Python Virtual Machine. similar to how multiple processes in a system share a single CPU.
- Many programs may be in memory, but only one is live on the CPU at any given moment.
- Likewise, although multiple threads may be "running" within the Python interpreter, only one thread is being executed by the interpreter at any given time.
- Access to the Python Virtual Machine is controlled by the global interpreter lock (GIL). This lock is what ensures that exactly one thread is running.

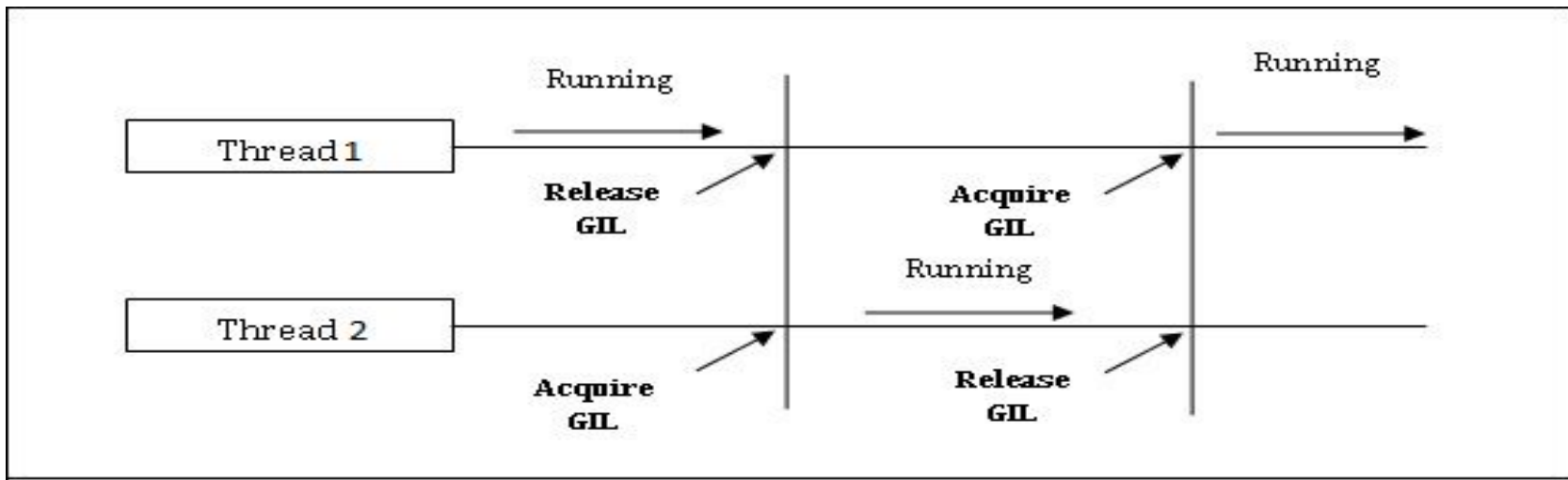
- This essentially means is a process can run only one thread at a time. When a thread starts running, it acquires GIL and when it waits for I/O, it releases the GIL, so that other threads of that process can run.

For example

Let us suppose process P1 has threads Thread1 and Thread2.

Thread1 running (acquire GIL) -> **Thread1** waiting for I/O (releases GIL)

-> **Thread2** running (acquires GIL) -> **Thread2** waiting for I/O (releases GIL)



❑ Multithreading Modules:

Python offers two modules to implement threads in programs.

- **thread module** and
- **threading module**.

Thread Module(`_thread`):

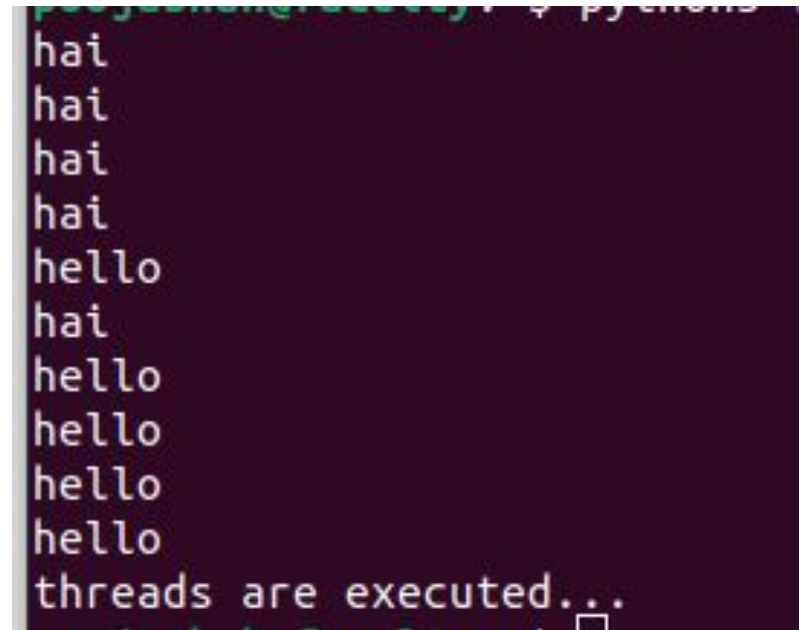
- This module provides low-level primitives for working with threads .
 - The `<_thread>` module supports one method to create thread. That is **`thread.start_new_thread(function, args)`**
- This method starts a new thread and returns its identifier. It'll invoke the function specified as the “function” parameter with the passed list of arguments. When the `<function>` returns, the thread would silently exit.
- Here, `args` is a tuple of arguments; use an empty tuple to call `<function>` without any arguments.

Note: Python 2.x used to have the `<thread>` module. But it got deprecated in Python 3.x and renamed to `<_thread>` module for backward compatibility.

Output:

Example:

```
from _thread import start_new_thread
from time import sleep
def disp(n):
    for i in range(5):
        print(n)
        start_new_thread(disp, ("hai", ))
        start_new_thread(disp, ("hello", ))
        sleep(2)
print("threads are executed...")
```



```
hai
hai
hai
hai
hello
hai
hello
hello
hello
hello
hello
threads are executed...
```


Threading Module:

- The threading module provides more features and good support for threads than thread module.
- This Module provides **Thread** class, and this **Thread** class provide following methods
 - **start()** – The start() method starts a thread by calling the run method.
 - **join([time])** – The join() waits for threads to terminate.
 - **isAlive()** – The isAlive() method checks whether a thread is still executing.
 - **getName()** – The getName() method returns the name of a thread.
 - **setName()** – The setName() method sets the name of a thread.

Creating Thread Using Threading Module

- To implement a new thread using the threading module, use following code snippet

Syntax:

threading.Thread (target=None, name=None, args=())

This method has following arguments. Those are:

- target** is the callable function to be invoked by the [run\(\)](#) method. Defaults to None, meaning nothing is called.
- name** is the thread name. By default, a unique name is constructed of the form “Thread-N” where N is a small decimal number.
- args** is the argument tuple for the function invocation. Defaults to ().

Example:

```
import threading
#function display
def display(msg):
    for i in range(5):
        print(msg)
# creating thread
t1 = threading.Thread(target=display, args=("Thread1",))
t2 = threading.Thread(target=display, args=("Thread2",))
# starting thread 1
t1.start()
# starting thread 2
t2.start()
# wait until thread 1 is completely executed t1.join()
# wait until thread 2 is completely executed
t2.join()
# both threads completely executed print("Done!")
```

Output:

```
>>>python
multithr.py Thread1
Thread1      Thread2
Thread1      Thread1
Thread2      Thread1
Thread2      Thread2
Thread2 Done!
```

Let us try to understand the above code:

- To import the threading module, we do:

import threading

To create a new thread, we create an object of Thread class. It takes following arguments:

target: the function to be executed by thread

args: the arguments to be passed to the target function

- In above example, we created 2 threads with different target functions:

*t1 = threading.Thread(target=display, args=("Thread1",)) t2 =
threading.Thread(target=display, args=("Thread2",))*

- To start a thread, we use start method of Thread class.

t1.start()

t2.start()

- Once the threads start, the current program also keeps on executing. In order to stop execution of current program until a thread is complete, we use join method.

t1.join()

t2.join()

- As a result, the current program will first wait for the completion of t1 and then t2. Once, they are finished, the remaining statements of current program are executed.

Example

```
import _thread
import time
def cal_sqre(num):
    print(" Calculate the square root of the given number")
    for n in num:
        time.sleep(0.3) # at each iteration it waits for 0.3 time
        print(' Square is : ', n * n)
```

```
def cal_cube(num):
    print(" Calculate the cube of  the given number")
    for n in num:
        time.sleep(0.3) # at each iteration it waits for 0.3 time
        print(" Cube is : ", n * n * n)
```

```
arr = [4, 5, 6, 7, 2]
```

```
t1 = time.time()
cal_sqre(arr)
cal_cube(arr)
```

```
Calculate the square root of the given number
Square is : 16
Square is : 25
Square is : 36
Square is : 49
Square is : 4
Calculate the cube of  the given number
Cube is : 64
Cube is : 125
Cube is : 216
Cube is : 343
Cube is : 8
Total time taken by threads is : 3.004389524459839
```